College of Computing

Lab: Collections – Part 1
Professor : Fahd KALLOUBI
Year : 2026/2027
Major : Computer Science (CS1)

Learning outcomes:

- Understand ArrayList and LinkedList collections
- Manipulating ArrayList and LinkedList using Iterator and ListIterator

Before starting this lab, go to my github account and then fork and clone the "lab6-problems" repository. I have created a package for each problem.

Problem 1:

you will implement a shopping cart using the ArrayList class. The file *Item.java* contains the definition of a class named *Item* that models an item one would purchase (this class was used in the previous lab). An item has a name, price, and quantity (the quantity purchased). The file *Shop.java* is an incomplete program that models shopping.

1. Complete Shop.java as follows:
    a. Declare and instantiate a variable *cart* to be an empty ArrayList.
    b. Fill in the statements in the loop to add an item to the cart and to print the cart contents (using the default *toString* in the ArrayList class). Comments in the code indicate where these statements go.
    c. c. Compile your program and run it.
2. You should have observed two problems with using the default printing for the cart object: the output doesn't look very good and the total price of the goods in the cart is not computed or printed. Modify the program to correct these problems by replacing the print statement with a loop that does the following:
    a. gets each item from the cart and prints the item
    b. computes the total price of the items in the cart (you need to use the *getPrice* and *getQuantity* methods of the Item class). The total price should be printed after the loop.
3. Compile and run your program.

Problem 2:

You'll need to create a simple banking application with a Customer and Bank type.

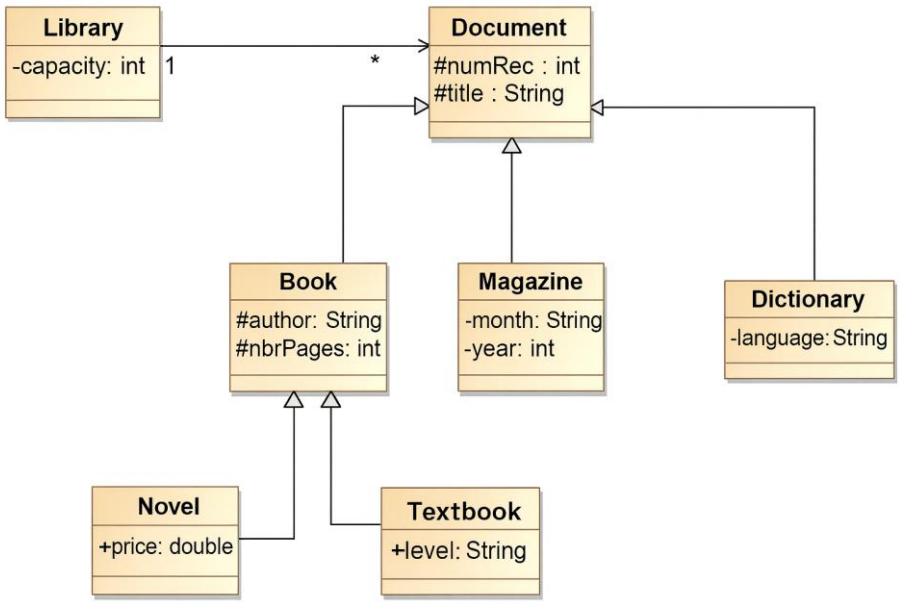
1. The Customer will have a name and an ArrayList of transactions containing Double wrapper elements.

    - A customer's transaction can be a credit, which means a positive amount, or it can be a debit, a negative amount.

2. The Bank will have a name, and an ArrayList of customers.
    - The bank should add a new customer, if they're not yet already in the list.
    - The bank class should allow a customer to add a transaction, to an existing Customer.
    - This class should also print a statement, that includes the customer name, and the transaction amounts. This method should use unboxing.
3. Give the corresponding class diagram of this description

4. Create a main class to test all the created methods.

Problem 3:



1. Develop the classes in the "problem3" package.
   - Respect the encapsulation principle,
   - Redefine the toString method in each class
   - numEnreg must be auto-incremental
2. A library will be represented by an ArrayList of documents. Define a Library class, with the following methods:
   - Library(int capacity) - constructor that creates a library with the specified capacity (maximum number of documents),
   - void displayDocuments() - displays all the books in the library,
   - boolean add(Document doc) - adds the specified document and returns true (false on failure),
   - boolean delete(Document doc) - deletes the specified document and returns true (false on failure)
   - Document document(int numEnrg) - returns the document with the record number provided as a parameter.
   - void displayAuthors() - displays the list of authors for all works that have an author.
3. Create a test class:
   - Create a library of n documents; the value of n will be read from the keyboard.
   - Initialize the library with two documents of your choice.
   - Then display a menu to select the type of operation (add, display, delete, etc.) with the option to exit.

Problem 4:

We want to Design and implement a program that compares the performance of ArrayList and LinkedList for a large number of:

- Random insertions/deletions in the middle of the list: Measure the time taken for add(index, element) and remove(index) operations.
- Sequential insertions/deletions at the beginning/end: Measure the time taken for addFirst(), addLast(), removeFirst(), removeLast() for LinkedList, and add(0, element), add(element), remove(0), remove(size-1) for ArrayList.
- Random access (get(index)): Measure the time taken to retrieve elements at various random indices.

To this end, complete the code of the ListPerformanceComparison class in package problem4. Then, Analyze the results and explain why one performs better than the other in each scenario based on their underlying data structures.

Problem 5:

First of all, try to visit the official documentation of the Stack class. Based on the documentation we can see:

> A more complete and consistent set of LIFO stack operations is provided by the `Deque` interface and its implementations, which should be used in preference to this class. For example:

```
Deque<Integer> stack = new ArrayDeque<Integer>();
```

And knowing that LinkedList is an implementation of Deque interface. We will implement a custom data structure for LIFO behavior called BookStack. Then, we will create another custom behavior for FIFO behavior called BookQueue.

1. I have defined the Book class for you
2. Create a BookStack class:
    a. Use a LinkedList<Book> internally, and map:
        - push() → addFirst()
        - pop() → removeFirst()
        - peek() → getFirst()
3. Create a BookQueue class:
    a. Use a LinkedList<Book> internally, and map:
        - enqueue() → addLast()
        - dequeue() → removeFirst()
        - peek() → getFirst()
4. Create a main class.

Here is the expected output:

```
=== Stack Test ===
Stack contents (top to bottom):
 - Clean Code by Robert C. Martin (2008)
 - 1984 by George Orwell (1949)
 - The Hobbit by J.R.R. Tolkien (1937)
Popped: Clean Code by Robert C. Martin (2008)
Top: 1984 by George Orwell (1949)
Stack contents (top to bottom):
```

```
 - 1984 by George Orwell (1949)
 - The Hobbit by J.R.R. Tolkien (1937)
=== Queue Test ===
Queue contents (front to back):
 - The Hobbit by J.R.R. Tolkien (1937)
 - 1984 by George Orwell (1949)
 - Clean Code by Robert C. Martin (2008)
Dequeued: The Hobbit by J.R.R. Tolkien (1937)
Front: 1984 by George Orwell (1949)
Queue contents (front to back):
 - 1984 by George Orwell (1949)
 - Clean Code by Robert C. Martin (2008)
```